



PAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

Programming in Practice (PAP521S)

Project A

Municipal Financial Management System

Technical Report

Group Members

FULL NAME	STUDENT NUMBER
Rumbidzai Tangwadzana	226002608
Shilongo Pinias	225114399
Johannes Nekongo	220064881
Uahorora kahitu	219140308
Oletu PN Hauwanga	226134512
Josef linkwangwa	226021297
Elisha Ajibola	220084599

Introduction

The Municipal Financial Management System (MFMS) is a menu-driven C application developed as the foundation of a municipal enterprise resource planning (ERP) system. The system was created to help a municipality manage important financial and administrative activities such as employee records, departmental budgets, suppliers, and municipal assets. This report explains the design and development of Project A, including the challenges we faced and how we solved them.

The project was developed using ANSI C (C99) in Visual Studio Code with the GCC compiler, while Git and GitHub were used for version control and collaboration. The system allowed us to apply the programming concepts we covered in Weeks 1–8, including variables, operators, decision-making, loops, arrays, strings, and functions

Problem Description

Municipalities deal with a lot of financial and administrative information on a daily basis. When this information is managed manually or using separate spreadsheets, it can lead to problems such as errors, wasted time, difficulty tracking budgets, and poor management of assets.

The main problem we identified is the lack of a simple and integrated system that can help municipal officials manage important information in one place. The system should allow officials to:

- Keep accurate employee records and calculate salary information.
- Manage departmental budgets and track expenditure.
- Register and search for suppliers.
- Keep track of municipal assets.
- Generate summary reports that can help with decision-making.

Project A was developed to address these problems by providing a basic C-based system that brings these functions together through a menu-driven interface. The system also includes input validation and clear output formatting to make it easier for users to work with the information.

System Objectives

The main objectives of Project A were to:

1. Develop a menu-driven C application that makes it easy for users to navigate between the different modules.
2. Use variables and suitable data types to store municipal information.
3. Allow users to enter information and validate the input to reduce errors and incorrect data.
4. Use arithmetic, relational, and logical operators to perform calculations and comparisons.
5. Use if, if-else, and switch statements to make decisions within the system.
6. Use loops to process multiple records and control how the program runs.
7. Use arrays to store and manage multiple employees, budgets, suppliers, and assets.
8. Use standard C string functions to work with text and string data.

9. Create functions to divide the program into smaller and more manageable sections.
10. Pass information to functions using parameters and return values where needed.
11. Organise the system into logical modules to make it easier to maintain and understand.
12. Work together as a team and use GitHub to manage our code and track our progress.

System Features

The MFMS consists of the following modules:

Main Menu

Provides navigation between all system modules and handles invalid choices.

Employee Management

- Add, display, and search employee records.
- Calculate gross salary.
- Validate salaries, allowances, and names.

Budget Management

- Enter budgets and expenditure for departments.
- Calculate remaining budgets.
- Identify departments that exceed their budgets.

Supplier Management

- Add, display, and search supplier records.
- Compare supplier information using string functions.

Asset Management

- Add, display, and search asset records.
- Store asset details such as ID, type, value, department, and condition.

Reports

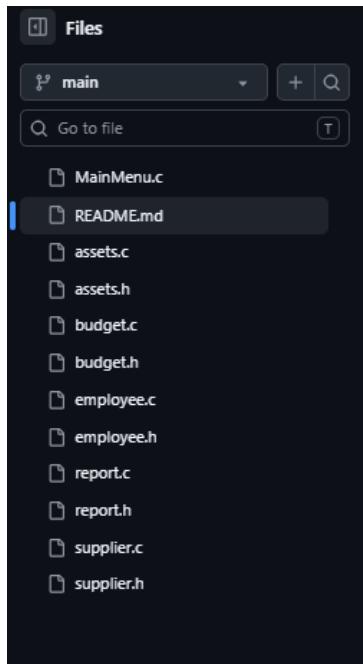
- Employee report with salary information.
- Budget report with expenditure and remaining budget.
- Supplier and asset reports listing registered records.

Input Validation

The system checks for invalid menu choices, negative values, empty names, and invalid numerical input.

Program Design

The system is divided into different source and header files to keep the code organised and make it easier to manage.



Challenges Encountered

1. **Modularisation** was one of the challenges we faced because the system was divided into different .c and .h files. We had to make sure that the function prototypes, shared variables, and include statements were correctly set up so that all the modules could work together without errors.
2. Git collaboration was also challenging because we were working as a team of seven members. We had to coordinate everyone's work, manage different branches, and deal with merge conflicts. We also had to make sure that the main branch remained functional while everyone was adding their own parts.

Solutions Implemented

1. To solve the modularisation problem, we separated the system into different .c and .h files based on each module. The header files were used for function prototypes and shared variable declarations, while the .c files contained the actual code for each module. This made the program more organised and easier to manage.
2. For Git collaboration, each team member worked on their own branch and committed their changes regularly. We then reviewed and merged the completed work into the main branch. We also communicated with each other when making changes to avoid conflicts and to make sure that the different modules could work together properly.

Conclusion

Project A successfully developed a basic Municipal Financial Management System in C and allowed us to apply the programming concepts covered in Weeks 1–8. The project also improved our technical skills in C programming, as well as our ability to use Git and GitHub for version control and collaborative development.

Besides the technical skills, the project also helped us develop important soft skills such as time management, teamwork, communication, and project management. Working on different modules as a team taught us how to divide tasks, manage deadlines, solve problems together, and make sure that everyone's work could be combined into one working system.

Overall, Project A gave us valuable practical experience that we can carry forward into future software development projects.